My Interactive Art Show

Final project brief, plan sheet and rubric (weeks 10–12)

Your mission

You are an artist who paints with code. In weeks 10, 11 and 12 you will make a Python turtle program that is a little **interactive art show**:

1. it says hello to the person watching (the “viewer”),
2. it asks them one or two questions,
3. it draws a picture or pattern that changes because of their answers,
4. and it has a surprise, so it is a bit different every time you run it.

In week 12 you will show it to your family at the **Showcase** and explain one part of your code. You can work alone or with your partner. If you work in a pair, you both need to be able to explain every part.

Tip: An example

Your teacher has a finished example called *Night Sky Postcard*. It asks “How many stars?” and “Day or night?”, then draws the sky, a moon or a sun, random stars and a row of houses. Ask to see it, then make something that is **yours**.

The six ingredients

Your finished program must use all six. Check off each one when it is in your code and working.

	Ingredient	What it looks like in Python
☐	**A question**	`answer = input("How many fish? ")`
☐	**A variable**	`fish_count = int(answer)` (a labeled box that remembers)
☐	**An `if`**	`if mood == "happy":` … `else:` …
☐	**A loop**	`for i in range(fish_count):` …
☐	**Something random**	`x = random.randint(-350, 350)`
☐	**My own function**	`def fish(x, y, color):` … and then calling it: `fish(0, 0, ORANGE)`

Idea menu

Pick one, mix two, or invent your own. Every idea can be an artwork or a little game.

Underwater world Asks “How many fish?” and “Deep or shallow?”. Your `fish(x, y, color)` function swims in random places.

Flower garden Asks “How many flowers?” and “Sunny or rainy?”. Rain draws lines, sun draws a sun.

Monster maker Asks “How many eyes?” and “Happy or grumpy?”. The `if` picks the mouth shape.

Space trip Asks “How many planets?”. Planets get random sizes; a `rocket()` function flies past.

Kaleidoscope Asks “How many sides?”. A loop turns and repeats your shape with random colors.

Snow day Asks “How much snow: a little or a lot?”. Snowflakes are made by a `snowflake()` function.

Fireworks show Asks “How many fireworks?”. Each firework is a burst of lines in a random place.

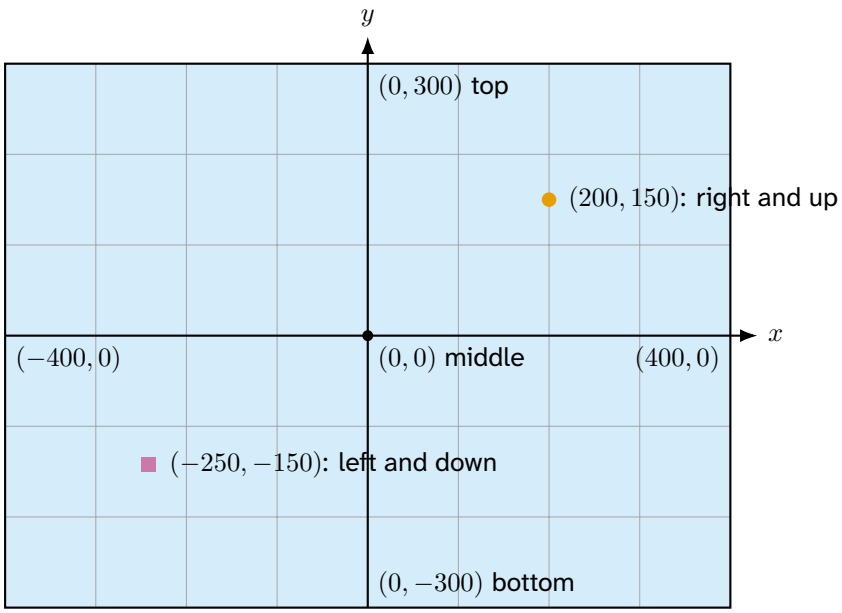
Guess-the-shape game The program secretly picks a random number of sides, draws the shape, asks “How many sides?” and the `if` says “Correct!” or “Try again”.

Digital citizenship & safety: Your art, your privacy

Use a nickname on your artwork, never your real full name, address or school. Do not ask the viewer for private things either: ask about colors, numbers, moods or animals instead. At the showcase we show projects on the screen only. Nothing is uploaded.

Where things go on the screen

The turtle window is `800` steps wide and `600` steps tall. The middle is $(0, 0)$. Use `t.penup()` and `t.goto(x, y)` to jump to a spot without drawing.



My plan sheet (Milestone 1)

Name (nickname is fine): _____ Date: _____

My project's name: _____

Sketch of my picture (draw it here; label the parts):

What my program asks (1 or 2 questions):

What it draws (and what changes because of the answers):

My function Name: _____ It needs: _____

(for example `fish(x, y, color)` needs a place and a color)

What it draws:

Where are my ingredients? Write a few words for each.

- ☐ My loop repeats: _____
- ☐ My `if` decides: _____
- ☐ My random surprise is: _____
- ☐ My variable remembers: _____

Teacher OK for my plan: _____

Milestones

Build in small steps. Each milestone has a check. Check it off when it is done.

	Milestone	Done when…
☐	**M1 Plan** end of week 10	My plan sheet is filled in and my teacher said OK. I opened `my_art_show.py` and saved it with my project's name.
☐	**M2 Core drawing works** week 11, first half	My function draws my main thing, and I call it at least twice in different places. The program runs without red errors.
☐	**M3 Surprise and polish** week 11, second half	My questions, my `if`, my loop and my random surprise all work. A partner tested it and gave me "two stars and a wish". I added a title or message.
☐	**M4 Showcase** week 12	I can run my program for a visitor and explain one part of my code in my own words.

Tip: Small steps beat big leaps

Change one thing, press **Run**, look. Then change the next thing. If something breaks, you know it was the last thing you changed. Save often with Cmd + S.

How my project is assessed

Your teacher uses this rubric. Use it yourself too: circle where you are now, and again at the end. Every column is a good place to be on the way.

I can…	Just starting	Getting there	Got it	Expert
make it my own (creativity)	I copied an example.	I changed colors or sizes in an example.	My picture has my own idea, my own function and a surprise.	My idea is unique and I added extra details or a twist nobody expected.
make my code work	It does not run yet.	It runs, but some parts are missing or broken.	It runs from start to end without errors.	It also copes with odd answers (like “lots” instead of a number).
use the six ingredients	I used 1 or 2.	I used 3 or 4.	I used all 6.	I used all 6, and my function has parameters I really use (like `x`, `y`, `color`).
explain my code	I can say what my program does.	I can point to my loop and my function.	I can explain how one part of my code works, line by line.	I can explain why I chose it, and how I fixed a bug.

Need a different path?

Tip: Support path: make it in Scratch

You can build the same art show in Scratch instead. The rubric is exactly the same, and you use the same six ingredients:

Ingredient	Scratch block
A question	**ask** … **and wait**, then the **answer** block
A variable	**Make a Variable**, then **set** / **change**
An if	**if** … **then** / **if** … **then** … **else**
A loop	**repeat** … or **forever**
Something random	**pick random** … **to** …
My own function	**Make a Block** (a My Block, with inputs if you can)

Stretch challenge: Extension path: react to key presses

Make your art show react while it is open. Put this **before** `turtle.done()`. Pressing the space bar adds a new star in a random place:

Key presses with onkey

```
def add_star():
    x = random.randint(-380, 380)
    y = random.randint(20, 280)
    t.penup()
    t.goto(x, y)
    t.dot(8, YELLOW)

screen.onkey(add_star, "space")   # no () after add_star
screen.listen()                   # start listening for keys
```

Click on the turtle window first, then press the key. Ideas: `"c"` changes the sky color, `"Up"` draws a firework, `"r"` adds rain.

Reflection: After the showcase

The part of my project I am proudest of:

One bug I found and how I fixed it: